REQ5 Stateful Creatures Design Rationale

FIT2099 Assignment 2

REQ5 introduces a stateful SporeBeast with four distinct states: Camouflaged, Foraging, Territorial, and Fleeing. The selected design uses the State pattern so that each state owns its normal turn behaviour, deterministic transition conditions, and entry effect while SporeBeast coordinates at most one transition per turn. This report consolidates the four member rationales into one design argument and explains why the composed State-pattern design is preferred over centralising all behaviour in the creature class.

1 Requirement Coverage and Overall Architecture

The common CreatureState contract gives every state the same lifecycle: createTurnAction creates the ordinary action, getNextState makes a deterministic transition decision, and onEnter performs a one-time effect when SporeBeast changes state. SporeBeast holds the current state through the interface, so it does not need concrete conditional branches for every state.

REQ5 rule	Whole-group design response	Primary responsibility
Four distinct states	CamouflagedState, ForagingState, TerritorialState, and FleeingState have separate implementations.	All Members
Deterministic transitions	State conditions inspect nearby actors, food availability, reachability, and hazards rather than use Random.	All state owners
Two exits plus staying	Each state design defines state-specific conditions for remaining active or returning to other appropriate states.	All state owners
Non-trivial transition effects	The lifecycle separates onEnter effects from ordinary actions, including food lure and spore-barrier effects.	All state owners
New involved behaviours	Foraging performs reachable-food search and Fleeing performs safe-route planning with hazard evaluation.	Peiyuncheng Ding and Zizhuo Wang
State controller	SporeBeast performs one transition decision and invokes onEnter only after the state object changes.	Haoze Hou

2 Framework and Camouflaged State

Haoze Hou provides the State-pattern framework and the initial CamouflagedState. Camouflage holds the creature still while it evaluates nearby threats and counts uninterrupted safe turns. The framework is the most important shared decision because every later state depends on its transition timing and abstraction boundary.

Design 1 Not Selected

Keep all state logic, transition conditions, counters, and entry effects in SporeBeast.playTurn using conditional statements and integer flags.

Advantages	Trade offs
The initial control flow is visible in one file and needs fewer classes.	SporeBeast would own actor data, action generation, every transition rule, and every entry effect, violating SRP.

A short prototype can be written quickly.	Each new state or condition requires editing the central creature class, violating OCP and making playTurn increasingly difficult to test.
No interface or state objects are required at first.	Concrete branches couple the context to every state implementation and make it easy to trigger more than one transition or repeat an entry effect.

Design 2 Selected

Define CreatureState and implement each state as a concrete class. SporeBeast stores a CreatureState reference, requests one next state, calls onEnter only after a change, and then requests the ordinary action.

Advantages	Trade offs
Each state owns one behavioural mode and can be read or changed independently.	The architecture introduces several classes, so developers must understand the common lifecycle before changing a state.
A fifth state can implement CreatureState without changing SporeBeast, supporting OCP.	The contract must stay focused; an overly broad CreatureState interface would force empty implementations.
SporeBeast depends on the abstraction rather than concrete states, supporting DIP and LSP.	A shared lifecycle means state authors must respect transition and entry-effect timing consistently.

Decision. Design 2 is selected. It keeps SporeBeast as the state-machine context, gives each state a focused responsibility, and enforces one transition per turn through a single framework location. CamouflagedState uses deterministic nearby-actor checks and a safe-turn count before handing control to the appropriate state.

3 Foraging State

Peiyuncheng Ding implements purposeful food seeking. The creature consumes eligible carried food, picks up eligible food at its current location, or finds the nearest reachable food and moves one legal step towards it. Threats take priority over food and can transfer control to territorial or fleeing behaviour.

3.1 Food Selection and Consumption

Design 1 Not Selected

Place concrete food checks and fixed healing behaviour directly in ForagingState after the creature reaches a target.

Advantages	Trade offs
The food rule appears in one method and is easy to follow for a small prototype.	Every added food type changes ForagingState, and fixed healing duplicates or loses existing food effects such as poisoning and maximum-health changes.
Few helper classes are needed.	Eligibility, pickup, and consumption become coupled, making stale-item checks and reuse harder.

Design 2 Selected

Use EdibleFoodFilter for eligibility, PickUpFoodAction for transfer, and the existing ConsumeAction and Consumable contract for food effects. The default diet accepts portable ConsumableProduce items.

Advantages	Trade offs
Eligibility, collection, and consumption have separate responsibilities, supporting SRP.	Legacy food that requires stamina is intentionally excluded because the health-only SporeBeast cannot safely consume it.
Existing food effects are preserved rather than reimplemented, avoiding DRY violations.	An extended diet still needs to verify that each accepted item honours the Consumable contract.
An injected predicate allows a future creature to use a different diet without rewriting movement.	The configurable policy adds one collaborator to the state constructor.

Decision. Design 2 is selected because it reuses the project's food behaviour and keeps the meaning of edible consistent across searching, pickup, and consumption.

3.2 Nearest Reachable Food and Movement

Design 1 Not Selected

Choose food by coordinate distance or wander until an edible item is encountered.

Advantages	Trade offs
Coordinate comparison is inexpensive and requires little infrastructure.	The closest coordinate may be blocked or occupied, so it does not guarantee a legal path or purposeful progress.
Wandering is simple to implement.	Random or incidental wandering does not satisfy the requirement for involved, deterministic food-seeking behaviour.

Design 2 Selected

Use NearestEdibleFoodFinder and DeterministicPathfinder with breadth-first search over enterable exits on the current map. Neighbours use a stable coordinate order.

Advantages	Trade offs
BFS finds the closest food by legal movement steps and excludes blocked or occupied tiles.	Target search and route search both use BFS, creating repeated work on larger maps.
Stable ordering makes equal-distance choices reproducible for testing and demonstrations.	Caching a route could improve efficiency later, but would need invalidation when food or actors move.
Current-map filtering prevents a cross-map exit from being mistaken for a local route.	The design assumes unweighted map movement, which is appropriate for the present game.

Decision. Design 2 is selected because it models the engine's legal movement rules rather than assuming geometric distance. Re-evaluating each turn avoids stale targets when another actor removes food.

4 Territorial State and Spore Traps

Chengzhang Su implements territorial response, including chasing, attacking, and local spore-trap behaviour. The state uses nearby-actor information to decide whether to attack, chase, or remain inactive, while the trap hierarchy keeps discardable environmental effects compatible with the engine Item model.

4.1 Territorial State and Chasing

Design 1 Not Selected

Create concrete class of Territorial State without extends from any super class or implements any interface, construct own methods with same parameters as the other states. Create concrete class for chase action with no inheritance from any super class, stores all the entities and methods inside the ChaseAction class. For the actor that can perform the chase action, direct execute the chase action effect inside the actor class.

Advantages	Trade offs
A small first version appears direct because all local effects are written in one class.	Repeated method signatures and no common contract increase coupling and reduce polymorphic substitution.
Concrete chase code is easy to find initially.	An action outside the engine Action hierarchy cannot be returned or executed through the normal action flow.

Design 2 Selected

Create concrete class of Territorial State that implement the existing CreatureState class, override the necessary methods and construct own implementation. Create concrete class for chase action inherit from Action super class, override the needed entities and methods inside the ChaseAction class. For the actor that can perform the chase action, create an interface Chasable maintaining the chase method, let the SporeBeast class implement this interface.

Advantages	Trade offs
The state participates in the shared lifecycle, and the action participates in normal engine execution.	A change to the chase algorithm may require editing or subclassing ChaseAction because the current chase policy is fixed.
ChaseAction depends on Chasable rather than SporeBeast, allowing other creatures to become chasers.	Developers must understand the engine Action contract as well as the Chasable capability.
CreatureState and Chasable are small focused interfaces, supporting ISP and DIP.	The extra abstractions add classes compared with a one-off concrete implementation.

Decision. Design 2 is selected because instead of directly referencing the territorial state object, adding an abstract contract can reduce the class coupling and dependency (Dependency Inversion Principle). Especially when several states containing same methods, but different implementations, interface will be a better choice than concrete or abstract class, for better extensibility (Open-Closed Principle). Since territorial state should only managed the effects when reached to the current state, create concrete class to implement interface to take over the effects (Single Responsibility Principle), or responsibility will be overlapped across classes.

4.2 Trap Hierarchy

Design 1 Not Selected

Create a concrete class for trap without extend from the Item super class from engine package, rewrite all the entities and behaviors of a trap inside the Trap class. For a spore trap, create a new method to hold the spore effect. Create an interface Discardable holding the method to discard a trap and create a concrete class DiscardAction directly hold the trap.

Advantages	Trade offs
The trap implementation is initially self-contained.	A non-Item trap cannot be placed, displayed, or passed through the engine's item pathways without duplicated plumbing.
Special spore behaviour can be written directly in one concrete trap.	New trap effects lead to type branches or edits to an existing class, and copied engine behaviour increases maintenance risk.

Design 2 Selected

Create an abstract class for trap extends from the Item super class from engine package, override the entities and behaviors of a trap inside the Trap class. For a spore trap, create an abstract method to hold the additional effect in Trap class, SporeTrap extends from super class Trap and override the additional effect method. Create an interface Discardable holding the method to discard a trap and create a concrete class DiscardAction directly hold the discardable trap.

Advantages	Trade offs
Trap reuses Item lifecycle and display behaviour, while each concrete trap owns only its special effect.	A simple trap with no extra effect may need an intentionally empty implementation of the abstract hook.
A future FireTrap can extend Trap without changing existing trap or action classes, supporting OCP.	The relationship between a trap's item presence and its effect must be tested carefully in the engine action flow.
DiscardAction depends on a capability interface, not a concrete trap type.	The hierarchy adds an abstraction layer for a small current family of traps.

Decision. Design 2 is selected to follow Liskov Substitution Principle, as any changes in the Trap class may affect the operations in DiscardAction class. The Discardable interface is holding the agreed contract between high- and low-level module, as long as the interface remain, the changes in modules won't affect the operation. For inheritance of the Trap from Item class, it is both to reduce code redundancy for better extensibility, like override the methods and implement own operations (Open-Closed Principle), and obey to Liskov Substitution Principle, where the trap can act as an Item to perform the Item behaviors or upcasting safely when needed.

5 Fleeing State

Zizhuo Wang implements deterministic escape behaviour. FleeingState separates sensing, safety checks, and route selection so that a change to a hazard rule does not require changing state control. On entry, the state deploys a spore barrier on valid neighbouring tiles; on ordinary turns, it moves along the best safe reachable route or returns a no-op action if no safe route exists.

Design 1 Not Selected

Place threat scanning, safety checks, route selection, transition rules, and trap placement directly in SporeBeast.playTurn.

Advantages	Trade offs
Fewer source files and a short initial path to a working prototype.	SporeBeast becomes actor, state controller, scanner, hazard evaluator, and route planner, violating SRP.
All logic is visible in one method during early experimentation.	Coordinates, threat radii, safety checks, and route ranking become tightly coupled and difficult to test independently.
No collaborators need to be constructed.	Adding a new hazard or escape policy requires editing the central creature class and risks breaking other states.

Design 2 Selected

Implement FleeingState as a CreatureState composed with ThreatScanner, SafetyEvaluator, and EscapeRoutePlanner. Route selection uses breadth-first search and stable tie-breakers.

Advantages	Trade offs
Sensing, safety rules, and route planning change independently, supporting SRP.	More classes must be understood to trace a complete fleeing turn.
Deterministic tie-breakers make escape routes reproducible for tests and demonstrations.	BFS considers more locations than an adjacent-tile heuristic, though the current maps are small.
SafetyEvaluator can be reused by future creatures that need to avoid fire and charged ground.	The entry effect relies on shared SporeTrap behaviour, so that item contract must remain compatible.
Constructor collaborators allow controlled replacement in focused tests.	The dependency structure is more verbose than direct calls in a single class.

Decision. Design 2 is selected. No nearby threat leads to CamouflagedState; exactly one adjacent threat leads to TerritorialState; otherwise the creature remains fleeing. This produces two deterministic exits plus a staying condition, while onEnter deploys the one-time spore barrier.

6 Cross State Design Principles

Principle	Application across REQ5
Single Responsibility	SporeBeast coordinates the lifecycle; each state owns one behaviour mode; focused helpers own food filtering, route planning, threat scanning, safety evaluation, chasing, or trap effects.
Open Closed	New state classes implement CreatureState; new chase-capable actors implement Chasable; new traps extend Trap; and new food policies can be supplied without rewriting the state framework.
Liskov Substitution	Every concrete state is usable through CreatureState, every chase action is usable through Action, and every trap remains usable through the engine Item abstraction.
Interface Segregation	CreatureState, Consumable, Chasable, and Discardable express focused capabilities instead of forcing unrelated behaviour on a class.
Dependency Inversion	SporeBeast depends on CreatureState; actions depend on capability interfaces; and state helpers are composed behind focused responsibilities rather than concrete game-object branches.
DRY and KISS	Shared lifecycle and engine actions prevent copied state-flow code. BFS is chosen for readable, deterministic search on the present unweighted maps rather than a more complex algorithm.
Connascence	Threat radius, safety criteria, food eligibility, and route ordering are encapsulated in named collaborators, reducing the chance that multiple classes must change together.

7 UML Sequence and Implementation Alignment

The REQ5 UML diagrams show Application creating StackQueueValleyWorld, the world placing SporeBeast, and SporeBeast holding the current CreatureState. The state classes realise the common interface and delegate specialised work to helpers such as EdibleFoodFilter, NearestEdibleFoodFinder, DeterministicPathfinder, ThreatScanner, SafetyEvaluator, EscapeRoutePlanner, Trap, and ChaseAction. This mirrors the source-code responsibilities described in this rationale.

The sequence diagrams show the common runtime order: the game invokes SporeBeast.playTurn; the current state determines one next state; SporeBeast runs the new state's entry effect only when the reference changes; and the active state returns one ordinary engine Action. The ordering prevents repeated entry effects while making state decisions and normal actions independently explainable.

8 Verification Strategy

Scenario	Expected result	States or components
Ten safe turns	Camouflaged state reaches its deterministic foraging transition condition.	CamouflagedState and SporeBeast
Nearby food with a blocked shortcut	Foraging chooses a legal reachable route rather than the closest coordinate.	ForagingState, finder, pathfinder
Food removed before pickup	Pickup rechecks current location and item availability instead of duplicating the item.	PickUpFoodAction
One nearby threat	The transition policy selects territorial behaviour rather than fleeing.	ForagingState and TerritorialState

Two nearby threats	The transition policy selects fleeing and runs the fleeing entry effect once.	ForagingState, FleeingState, SporeBeast
Fire or charged ground on escape route	Safety evaluation rejects the hazardous route even when it is farther from threats.	SafetyEvaluator and planner
No safe escape route or map edge	Fleeing returns DoNothingAction without an exception.	FleeingState and planner
Adjacent target outside attack range	Territorial state returns the appropriate chase or attack action through the engine Action model.	TerritorialState and ChaseAction
Trap encounter and discard	The trap is treated as an Item and its effect is reached through the discard capability workflow.	Trap, SporeTrap, DiscardAction

9 Conclusion

The selected REQ5 design uses a composed State pattern to keep the four creature behaviours distinct without duplicating game-loop logic. The design makes transitions deterministic, assigns entry effects to the transition lifecycle, and reserves involved behaviours for food seeking and safe route planning. The main trade-off is a larger set of collaborating classes, but the resulting cohesion, extensibility, and testability are preferable to a central conditional implementation inside SporeBeast.